event_data_workflow

How raw event-camera recordings become GPU-ready training batches — the story of what broke, what got fixed, and what's still open

SNNs auf GPUs · branch `46-cache_engine` · August 2026

Contents

1. 1 What this folder is for
2. 2 The story so far
3. 3 How the pieces fit together
4. 4 File by file
5. 5 The measurements — hard numbers, not impressions
6. 6 What's working well
7. 7 What's still falling apart
8. 8 Where to push next

1. What this folder is for

`event_data_workflow` is not the SNN itself — it is the plumbing in front of it. Event cameras don't record images; they record sparse streams of "this pixel just got brighter/darker" events. Before any of the training code in `learning/` can see that data, it has to be loaded, converted into dense picture-like frames, cached so that work isn't repeated, sliced into time windows, batched, and physically copied onto the GPU. This folder is that entire supply chain.

It matters more than its size suggests, because everything the project ultimately wants to compare — accuracy across frameworks, energy per sample, how well different hardware is used — sits on top of this plumbing. If the plumbing quietly does the wrong thing, every comparison built on top of it inherits that error silently. That is exactly what was found and fixed here, described below.

2. The story so far

It started with a simple question: when event-camera data gets turned into something the GPU can train on, is the GPU actually busy the whole time — or is it sitting there waiting?

Surprise one: it crashed, it didn't just stall

The first attempt at a real training pass to check this didn't slow down — it crashed outright with a cryptic Windows error. The crash had nothing to do with GPUs or training: it happened in the step where recordings get saved to disk-cache files, using a compression option that is broken on this particular combination of Python and the `h5py` library on this machine. A tiny, isolated script that had nothing to do with the project — "save a file with compression on" — crashed the same way, proving it was an environment quirk, not a bug in this codebase. Fix: turn that compression option off.

Surprise two: the cache wasn't caching the expensive part

Once the crash was gone, something odder showed up: asking for the same sample twice was still slow the second time, as if it were being reprocessed from scratch. The caching step was saving the raw recording, but the expensive part — converting the raw event stream into the dense frames the model actually trains on — was happening fresh on every access regardless of the cache. The fix rewired the caching classes so the expensive conversion happens once and is reused. Verified directly: the same sample, asked for twice, dropped from ~50 ms to ~2 ms on the second ask, with byte-for-byte identical output — not just faster, still correct. This is the `PreTransformedDataset` pattern described in Section 4.

Building a way to actually see the problem

With both bugs fixed, the next question was: how much is the GPU actually idle, honestly measured, not guessed? That led to a small measuring tool — the "GPU utilization harness" (`diagnostics/gpu_utilization_harness.py`). For each batch it times three things separately: how long it takes to get data ready (**fetch**), how long it takes to move that data onto the GPU (**transfer**), and how long the GPU spends actually computing on it (**compute**). At the same time, a background thread — `PipelineMonitor`, in `system_monitor.py` — asks the GPU driver roughly 20 times a second how busy it is, tagging every answer with whichever phase was active at that instant. That's what produces the colored timeline charts in Section 5.

Closing the gap: prefetching and compiled replay

Two ideas, drawn from what PyTorch itself recommends, from NVIDIA's own data-loading library, and from published work on this exact "GPU idles waiting for data" problem, turned out to matter most:

- Instead of fetch → transfer → compute happening strictly one after another, start moving the *next* batch onto the GPU while the current one is still computing — a relay handoff instead of a stop-and-go queue. This is `AsyncGPUPrefetcher` and `CudaPrefetcher` in `prefetch.py`.
- The model processes each sample as 16 tiny sequential timesteps, each normally requiring its own little CPU↔GPU back-and-forth. `torch.compile` records that sequence once and replays it as a single instruction, skipping most of the repeated overhead.

Implemented together and re-measured, one batch dropped from ~350 ms to ~135 ms — roughly 2.6× faster end to end — and the "waiting on data" portion specifically fell over 100× smaller, down to a couple of milliseconds. Section 5 shows this is not a rounded-off approximation; it's what the harness's own logged CSVs say.

Being honest about what didn't fully work

GPU utilization *while it had work queued* improved but stayed well under 100% — the leftover piece of the "16 tiny timesteps" problem that compiled replay only partly solves. Keeping the *entire* dataset resident in GPU memory ahead of time was checked and ruled out mathematically: it doesn't fit safely, and forcing it would cause constant eviction — worse, not better. A lighter version — several batches queued in VRAM at once instead of one (the `depth` parameter on `CudaPrefetcher`) — was built and tested, but didn't move the needle further, because the fetch-side bottleneck it would have addressed was already solved by the earlier fix. Both of these negative results are kept in this report rather than quietly dropped, because a report that only shows the wins isn't trustworthy.

"Not your code, an environment quirk" and "still slow, but correct" turned out to matter more than any single optimization — both bugs would have silently distorted every comparison built on top of this pipeline.

3. How the pieces fit together

Cache selection must always happen on the *raw* recording, before any temporal slicing — one recording produces many time-slices, so caching the recording once yields many cache hits from a single stored entry. Applying the cache after slicing is treated as a programming error and raises immediately.

```
SystemResourceMonitor  (cuda_enabled-gated RAM / disk / VRAM probe)
        |
        +--> dataloader_config()                    [data_pipeline.py]
        |         num_workers / pin_memory / prefetch_factor,
        |         sized from live RAM headroom
        |
        +--> AdaptiveCacheController.determine_dataset_strategy()
                  |
                  +-- MemoryCachedDataset     (RAM,    small dataset)
                  +-- DiskCachedDataset       (disk,   large dataset)
                  +-- BoundedRecordingCache   (hybrid: RAM hot layer over disk)
                  +-- GPURecordingCache       (VRAM-resident, GPU-only runs)
                           |
                           v
                  DataLoader --> AsyncGPUPrefetcher --> CudaPrefetcher --> training loop
                                 (background thread)     (H2D copy on
                                                          its own CUDA
                                                          stream, N
                                                          batches deep)
```

4. File by file

`cache_engine.py` actively changing

487 lines · the decision-maker: which cache tier gets used, and the four tiers themselves

`AdaptiveCacheController` looks at live RAM, disk, and VRAM headroom and picks one of five strategies per dataset split: `memory`, `disk`, `hybrid`, `gpu_memory`, or `no_cache` — or a forced choice via `force_mode` in config. **`GPURecordingCache`** is the newest tier: recordings are encoded once and kept resident as CUDA tensors, with a per-training-phase VRAM budget (`GPU_PHASE_CAPS` — tighter during backward/warmup, looser during eval) and an emergency-eviction path that shrinks the cache if free VRAM drops too low. The deterministic/stochastic transform split (`transform` vs. `live_transform`) that fixes the augmentation-freezing bug (Section 6) lives here. Currently uncommitted on this branch — this is where most of the GPU-cache and phase-aware-budget work landed.

`system_monitor.py` actively changing

294 lines · resource probing (point-in-time) and continuous monitoring (over time)

`SystemResourceMonitor` answers "how much RAM/disk/VRAM is free right now" for cache and worker-count decisions, gated by an explicit `cuda_enabled` flag so a GPU that physically exists but wasn't requested never influences the answer. **`PipelineMonitor`** is new: a background-thread sampler (~20 Hz) that records CPU%, RAM, GPU utilization/power/clock over the whole run, tagged with a caller-set phase label, and flags sustained "out of bounds" episodes (GPU idle too long, CPU pegged, RAM too low) without ever touching a CUDA tensor itself — so measuring never perturbs the thing being measured. This is what the diagnostics harness in Section 5 is built on.

`prefetch.py` **actively changing**

124 lines · keeps the GPU from waiting on the CPU

`AsyncGPUPrefetcher` wraps a DataLoader and runs it one batch ahead in a background *thread* (not a process — CUDA state doesn't survive a fork/spawn boundary cleanly). `CudaPrefetcher` goes further: it issues the host→device copy for the next `depth` batches on a dedicated CUDA stream while the current batch is still computing on the default stream, so the copy is always done ahead of when it's needed rather than issued just-in-time. `depth` batches sit resident in VRAM at once — cheap, a few MB each — but as documented in Section 2, pushing depth past 1 didn't produce a further measurable win once the fetch bottleneck itself was gone.

`data_pipeline.py` **stable**

424 lines · the orchestrator — wires everything above into train/test DataLoaders

`NeuromorphicEncoder` is the one class training code actually instantiates: load raw recordings → decide dataset shape → apply the cache strategy → optionally slice into temporal windows → build DataLoaders. Also home to `dataloader_config()` (worker count sized from live RAM headroom — drops to `num_workers=0` under ~500MB budget for GPU-only embedded runs) and `create_sliced_dataset()` (temporal windowing via tonic's `SlicedDataset`). Both were folded in from a since-deleted `pipeline_coordinator.py/temporal_slicer.py` pair — see Section 7 for why that split happened. Also holds the built-in dataset registry (N-MNIST, N-Caltech101, ASL-DVS, DVS128 Gesture, and DSEC for the regression track) and the interactive picker used when no dataset is preconfigured.

`regression_datasets.py` **stable, small edit pending**

85 lines · the one dataset that doesn't fit the standard shape

`DSECRaw` exists because tonic's own DSEC loader hands back one whole recording (~134 million events) as a single sample — unusable directly, either as a cache unit or a batch. DSEC's own frame-timestamp metadata is used to slice each recording into (event window, matching optical-flow frame) pairs, turning one recording into N proper training samples. Forces single-process DataLoader loading, because its underlying HDF5 file handles cannot safely cross a worker-process boundary.

`gpu_stats.py` stable

152 lines · per-epoch GPU utilization/memory/power bookkeeping for training runs

Separate from `PipelineMonitor` — this one is scoped to a training epoch rather than an arbitrary phase-tagged window, and additionally tracks peak VRAM and (optionally) idle-baseline- subtracted power draw so energy-per-epoch numbers reflect the workload's actual cost rather than the GPU's resting draw. Degrades to a complete no-op on a CPU-only machine or one without NVML, by design.

`workflow_config.py` stable

32 lines · loads `configuration/data_workflow.yaml` into typed settings

Frame mode (time-window vs. fixed timestep count), temporal-slicing on/off and duration, cache path and thresholds, and `force_mode` — the single adaptive on/off switch for the whole cache-tier decision (deliberately not a second boolean layered on top; see Section 6).

`__init__.py` stable

3 lines · public surface: `NeuromorphicEncoder`, `resolve_dataset_entry`, `DATASET_REGISTRY`

Everything else in this folder is an implementation detail from the outside — `main.py` only ever imports through here.

5. The measurements — hard numbers, not impressions

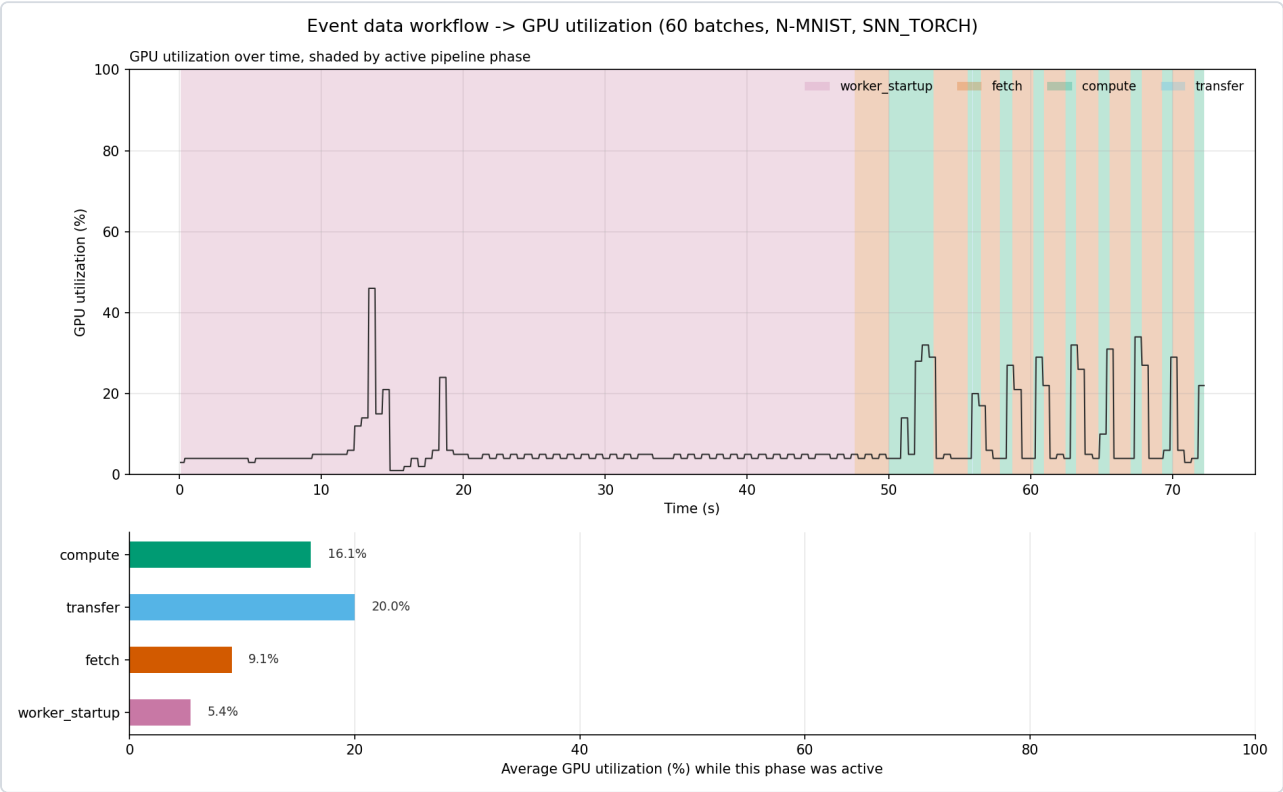
All figures below come directly from `diagnostics/gpu_utilization_harness.py`'s own logged CSVs (steady-state average across 59 batches, discarding batch 0 as a one-time warmup/compile outlier), not from eyeballing the charts.

Run	Fetch (avg)	Transfer (avg)	Compute (avg)	Total / batch	vs. baseline
Baseline (cache fix only, no prefetch/compile)	216.5 ms	1.6 ms	131.7 ms	**349.8 ms**	—
Optimized (prefetcher + `torch.compile`, default mode)	2.1 ms*	fused*	132.8 ms	**134.9 ms**	2.59× faster
Optimized, prefetch depth = 8	1.9 ms*	fused*	136.6 ms	**138.5 ms**	2.53× faster

Abandoned: CUDA-graphs (`mode="reduce-overhead"`)	—	~5,375 ms	**~6,020 ms**	17× *slower*

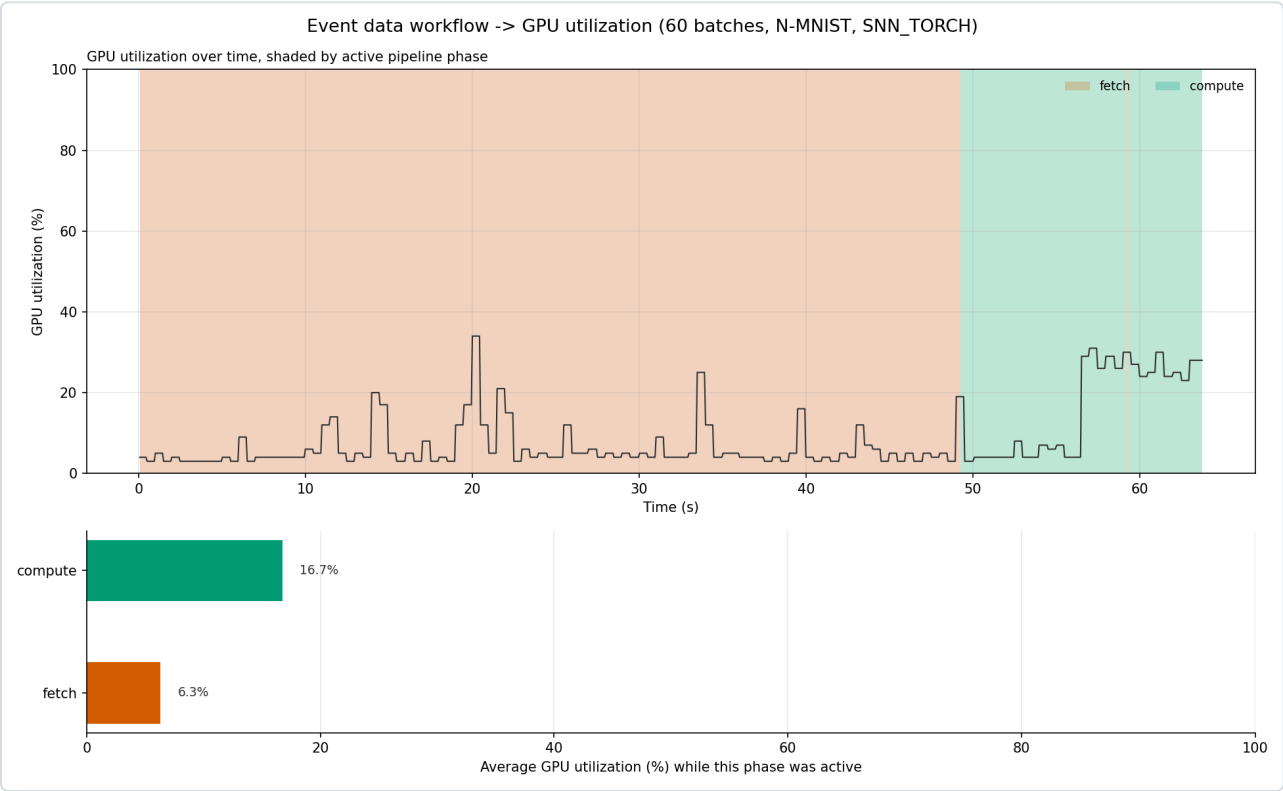
* In optimized mode, `CudaPrefetcher` fuses fetch and the host→device transfer into one measured phase, since the transfer for a batch is issued ahead of time on a side stream while the previous batch is still computing.

The abandoned row is not a hypothetical — it's the actual logged result of the one CUDA-graph experiment that was tried and reverted (also referenced directly in `learning/main.py`'s comments). SNNTorch's internal neuron state changes tensor rank across timesteps, which repeatedly blows CUDA graphs' "fixed, replay-safe kernel sequence" assumption — dynamo recompiles constantly instead of replaying, and the batch time explodes rather than shrinking. It's kept in this report as a documented dead end, not deleted from history.



Baseline run. The wide pink band is one-time DataLoader worker-process startup (~47s of the ~72s total for just 60 batches) — a real cost, but a one-time one, not a per-batch tax. Once past it, orange (fetch) and green (compute) bands alternate roughly evenly.

Optimized run. Fetch and transfer are fused (Section above); the compute band visibly widens near the end as `torch.compile`'s replayed graph starts paying off and GPU utilization climbs into the high-20s%, still well short of saturation.



Optimized run, prefetch depth = 8. Visually near-identical to depth = 1 — the direct evidence behind Section 2's honest negative result: a deeper VRAM-resident queue doesn't help once the fetch-side bottleneck is already gone.

6. What's working well

Verified, not assumed

- **The cache-reuse fix is proven correct, not just faster.** Same sample, asked for twice: ~50ms → ~2ms, byte-for-byte identical output.
- **The augmentation-freezing bug is fixed and independently verified.** Deterministic transform (Denoise+ToFrame) now runs only on a cache miss; the stochastic transform (RandomRotation) runs fresh on every access and produces a different result each time — checked directly in `test_cache_engine_sample.py` (18/18 checks pass), for both the CPU cache tier and `GPURecordingCache`.
- **The fetch/transfer bottleneck is functionally solved.** 216.5ms → ~2ms average, over 100× smaller, confirmed by the harness's own logs (Section 5), not an estimate.
- **Resource probing correctly respects intent, not just hardware presence.** `SystemResourceMonitor`'s `cuda_enabled` flag means a physically-present GPU that a run didn't ask for never steers cache or worker decisions — a real bug that existed before and was fixed and tested.
- **Graceful degradation is consistent across the whole folder**, not bolted on in one place: no CUDA → `gpu_stats.py` and `PipelineMonitor` both become clean no-ops; no NVML → same; a non-interactive terminal that lies about being a TTY → `EOFError` is caught and both pickers fall back to sane defaults instead of crashing.
- **The math on "just cache the whole dataset in VRAM" was actually done**, not hand-waved — checked and correctly ruled out before being tried, saving an implementation effort that would not have paid off.

7. What's still falling apart

> **Open, acknowledged problems — not swept under the rug**
>
> - **GPU utilization during compute is still well under saturation.** Prefetching fixed the *waiting* problem; it did not fix the GPU doing enough work per timestep once data has arrived. The 16-timestep-per-sample structure still means a lot of small CPU↔GPU round trips inside compute itself — compiled replay only partly absorbs this. This is the one problem today's fixes were explicitly not meant to solve.
> - **The core changes are still uncommitted** on branch `46-cache_engine` — `cache_engine.py`, `prefetch.py`, `system_monitor.py` all show as modified-but-not-committed in the working tree as of this report. Everything described above is real and runnable, but not yet locked in as project history.
> - **Single-GPU assumption is a documented, real limitation**, not a hidden one: `SystemResourceMonitor` and everything built on it track VRAM for exactly one `device_idx`. A multi-GPU run would need one instance per device, or aggregation across devices — neither exists yet.
> - **Worker count is currently RAM-limited, not CPU-limited**, on machines running close to full memory: `dataloader_config()`'s adaptive sizing caps `num_workers` from available RAM headroom, so a machine with plenty of CPU cores but little free RAM silently gets few workers — correct behavior, but easy to misdiagnose as a CPU bottleneck if you don't know to check RAM first.
> - **Live augmentation is a permanent per-epoch tax by design.** `RandomRotation` can't be cached — it must vary — so it re-runs on every single access, every epoch, forever. Not measured yet exactly how much of "fetch" time this specific op accounts for.
> - **The CUDA-graphs / `reduce-overhead` attempt is a cautionary result, not a solved one** — ~17× slower, left disabled with an explanatory comment rather than deleted, in case the underlying SNNTorch state-rank issue is ever worth revisiting.

8. Where to push next

Roughly in order of how much each would actually move the needle, based on what the harness has already shown:

1. **Free RAM headroom** — directly unlocks more parallel DataLoader workers under the existing adaptive sizing logic. Not a code change; the current constraint on some

machines is memory, not CPU core count.

2. **Move part of the CPU-side work onto the GPU** — the harness shows the GPU has real spare capacity even during "compute." Denoise+ToFrame and RandomRotation are both currently CPU/numpy work; either could plausibly become a GPU tensor op, cutting CPU+RAM load per worker while using otherwise-idle GPU cycles. The biggest potential win, and also the most invasive — it touches the transform pipeline itself, not just orchestration around it.
3. **Measure exactly how much of "fetch" time RandomRotation costs** before deciding whether it's worth optimizing on its own.
4. **Bulk pre-warm the cache once, at max parallelism, before training starts**, instead of the current interleaved pattern where cache misses happen scattered throughout training. Doesn't reduce total CPU work, but moves it out of steady-state training time.
5. **Raising `num_workers` directly is the lowest-leverage, riskiest lever right now** — under tight free RAM, more workers risks swapping, which makes things slower, not faster.